Cloud Computing (CS3081)

Name : Lakshya Singh

Roll No. : 23010302

Github : https://github.com/Cloud_Assignment

1. Fundamental Learning & Conceptual Takeaways

Undertaking this mini project provided me with an end to end practical understanding of the interaction between ML systems and cloud infrastructure. It helped me to see how a machine learning model can be taken from a local development environment to a scalable, cloud based deployment where it can be accessed as a real service by users.

Rather than treating model training and service deployment as isolated tasks, this project demonstrated the complete lifecycle of an AI system in a cloud environment. It gave me hands-on experience in model development, containerization, deployment, and service management, while also highlighting key cloud concepts such as scalability, high availability, fault tolerance and cost awareness.

Overall, the project helped me build a clearer and more practical understanding of how cloud computing enables the reliable, efficient and large-scale serving of machine learning models.

(a) Sequence level knowledge distillation on consumer hardware, including its purpose, workflow and practical benefits such as model compression and faster inference.

(b) Cloud Native storage Decoupling & Containers Immutability, and how they help in building portable, consistent and easy-to-deploy ML services.

(c) Systems Automation & Stateless Network Security, including the use of automation (e.g., CI/CD) and best practices for securing cloud based services.

3. Empirical Findings, SRE Observations & What could be done better.

Key Empirical Findings & SRE observations.

1. Chaos Recovery & SRE observations : Deleting an active pod under load resulted in recovery within 18.93s with zero dropped connections. Traffic ingress evenly distributed traffic across replicas (53.4%/46.6%).

2. Economic serving cost : Under the project rate card, the dual replica deployment costs 7.9 INR/hr, serving 2052 requests/hr, result in a unit cost of 3.85 INR per 1000 requests.

What could have been done better.

1. Pre-hydrated persistent volumes : Using a shared ReadWriteMany volume with Safetensors, memory mapping (mmap = True) would eliminate network weight download, reducing cold starts from 18.9s to sub-second levels.

2. Stateful eBPF security policies : Replacing stateless NACLs with cilium eBPF network policies would allow connection tracking and identify anomaly filtering without managing ephemeral port return rules.

2. Technical challenges Encountered & Engineering Solutions.

Challenge 1 : OLLAMA cross-Containers Network Refusal in k3d cluster.

Solution : I reconfigured and launched the host OLLAMA service with OLLAMA_HOST = 0.0.0.0:11434, enabled OLLAMA_FLASH_ATTENTION = true, and set OLLAMA_NUM_PARALLEL = 2, enabling multiple worker pods to query concurrent inference slots without starvation.

Challenge 2 : SeaweedFS containers OOM Termination during model uploads.

Solution : I updated k8s/data/seaweedfs.yaml to provision a 2 GiB persistent VolumeClaim (seaweedfs.pvc) mounted under /data and raised the memory limit to 1.5 GiB, permanently resolving storage persistence and memory starvation.